Web Scraping for a RAG Pipeline

Case study: extracting main content from a Moroccan government page (mmsp.gov.ma)

---

1. What we were trying to do

The goal was to scrape a page from the Moroccan Ministry of Digital Transition website (mmsp.gov.ma) to extract clean article text for a RAG (Retrieval-Augmented Generation) ingestion pipeline. The initial symptom was that only the page headers and navigation menu were being captured, while the actual article content was missing from the output.

2. How we diagnosed the problem

The first hypothesis was that the page relies on JavaScript to render its main content client-side, which would explain why a simple HTTP request only returns an empty shell. This is a very common cause of this exact symptom, so it made sense to test it first with Playwright, a headless-browser automation library capable of executing JavaScript before extracting the page content.

However, after fetching the raw HTML directly, it turned out the main content was already present in the server-rendered HTML. This meant the issue was never about JavaScript rendering at all — it was a wrong CSS selector being used to locate the content in the page structure.

By inspecting the page's HTML directly, the real content was found to live inside a specific container:

```
<div id="read_content"> ... </div>
```

This div also contained some unwanted elements mixed in with the article text: a text-to-speech “Ecouter” button, and a script tag for a social-sharing widget (AddThis). Both needed to be removed before extracting clean text.

3. The final scraping function

```
import requests
from bs4 import BeautifulSoup

def scrape_mmsp_page(url):
    headers = {"User-Agent": "Mozilla/5.0 ..."}
    r = requests.get(url, headers=headers)
    soup = BeautifulSoup(r.text, "html.parser")
    content_div = soup.find("div", id="read_content")

    if content_div is None:
        return None

    junk_tags = ["button", "script", "style", "noscript", "svg", "iframe", "form"]
    for tag in content_div.find_all(junk_tags):
        tag.decompose()
    return content_div.get_text(separator="\n", strip=True)
```

4. Line-by-line explanation

`import requests`

Imports the library used to make HTTP calls — this is what downloads the raw HTML of a page, the same way a browser does when you type a URL.

`from bs4 import BeautifulSoup`

Imports the BeautifulSoup class, which turns messy raw HTML text into a structured, navigable object so you can search for tags by id, class, or name.

`def scrape_mmsp_page(url):`

Defines a reusable function that takes a single page URL as input, so it can be called again for any other page on the same site.

`r = requests.get(url, headers=headers)`

Sends an HTTP GET request to the URL and stores the response in r. Adding a custom User-Agent header makes the request look like it comes from a real browser, which avoids some servers serving a stripped-down or blocked page to non-browser clients.

`soup = BeautifulSoup(r.text, "html.parser")`

Parses the raw HTML string (r.text) into a tree structure stored in soup, using Python's built-in HTML parser.

`content_div = soup.find("div", id="read_content")`

Searches the parsed tree for the first div tag whose id attribute equals read_content — this is the exact container identified as holding the real article text.

`if content_div is None: return None`

A safety check: if no matching div is found on the page (wrong URL, different template, etc.), find() returns None. Returning early avoids a crash later when trying to work with something that doesn't exist.

`junk_tags = [...] / for tag in content_div.find_all(junk_tags): tag.decompose()`

find_all() collects every tag inside content_div matching any of the given names (button, script, style, noscript, svg, iframe, form). The loop then goes through that list one element at a time, and .decompose() permanently deletes each one from the tree. This removes non-article elements such as the text-to-speech button or embedded scripts before the text is extracted. If a page has none of these tags, find_all() simply returns an empty list and the loop does nothing — no error occurs.

`return content_div.get_text(separator="\n", strip=True)`

Extracts only the visible text remaining inside content_div, ignoring HTML tags. separator="\n" places a newline between separate text blocks (paragraphs, list items) instead of merging them together, and strip=True removes extra leading/trailing whitespace from each piece.

5. The actual bug: it was in the link

After writing the function correctly, testing it still returned None. The cause was not the scraping logic itself but the URL string used to call the function: it contained a leading and a trailing space.

```
url = " https://www.mmsp.gov.ma/fr/nos-services/...chafafiya "
```

Extra whitespace around a URL can cause the request to fail or to be handled unexpectedly by the server, resulting in a response that no longer contains the expected read_content div — which is exactly why content_div.find(...) returned None. Removing the spaces fixed the issue immediately.

This is also why adding debug prints is valuable when troubleshooting a scraper:

```
print(r.status_code)
print(len(r.text))
print("read_content" in r.text)
```

These three lines check, in order: whether the request succeeded (status code 200), whether a real page was returned (a reasonable content length), and whether the target element actually exists in the raw HTML received. This isolates whether a failure comes from the request itself, from the page content, or from the parsing logic.

6. Key takeaways for future scraping work

- Always check View Page Source (or fetch the raw HTML directly) before assuming a page needs a headless browser — many sites render content server-side.
- Use a specific id or class selector rather than a generic tag like main when possible; it is more reliable and less likely to accidentally include navigation or menus.
- Strip out non-content tags (button, script, style, noscript, svg, iframe, form) before calling get_text(), so extracted text stays clean for embedding/indexing.
- A function should fail gracefully (return None) rather than crash when an expected element is missing.
- When something inexplicably breaks, check the simplest possible cause first — stray whitespace in a string is a very common, easy-to-miss bug.
- For sites with unknown or inconsistent structures, trafilatura is a solid general-purpose fallback extractor, while BeautifulSoup with a custom selector remains the more precise choice once a reliable selector has been identified for a given site.